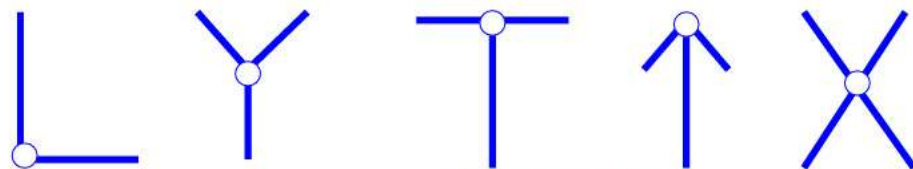


第5章 基元检测

- 概念
- 基元：泛指图像中有特点的基本单元，比如边缘、角点、直线段、圆、孔（及它们结合体）等。也称为特征，因此基元检测也称为特征检测

5.1 特征点的概念

- 与周围像素点具有不同特性的像素，比如角点（corner point）

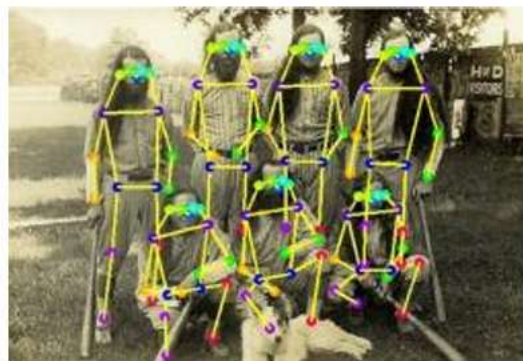


斑点 (blob) 中心



5.1 特征点的概念

- 三种相似的术语
 - 特征点: feature point, 统称
 - 兴趣点: interest point, 传统
 - 关键点: keypoint, 有时包含了一定的语义



5.1 特征点的作用

- 反映目标形状：通过特征点匹配可实现目标识别



模板图



待匹配图



5.1 特征点的作用

- 反映目标姿态：通过特征点匹配可分析目标姿态，进而判断其行为



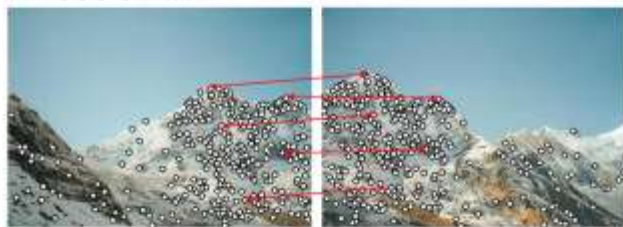
5.1 特征点的作用

- 进行人体姿态转换：通过特征点匹配实现人体的姿态转换



5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 在两幅图像中检测特征点
 - 寻找对应点



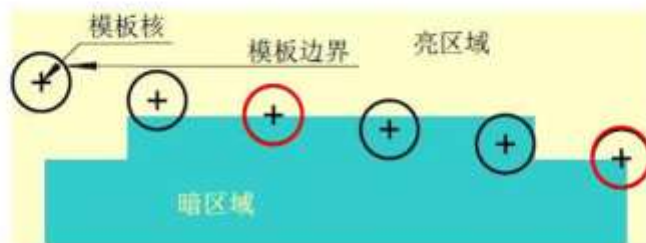
5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 在两幅图像中检测特征点
 - 寻找对应点
 - 利用点对计算两图间的变换关系并拼接图像



5.2 SUSAN算子——USAN

- 核同值区域（univalue segment assimilating nucleus, USAN），即与核有相同值的区域

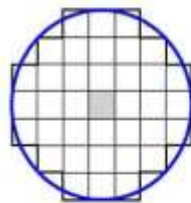


- 特点：
 - 可用于检测角点、边缘
 - 不需要计算微分，对噪声不敏感

理论上小于 $S_{max}/2$ 为角点，等于 $S_{max}/2$ 为边缘：对于离散模板而言，不满足

5.2 SUSAN算子——原理

- 设定模板
 - 通常取矩形模板



- 将模板内每个像素的灰度值与核的灰度值进行比较

灰度差阈值

$$C(x_0, y_0; x, y) = \begin{cases} 1 & \text{当 } |f(x_0, y_0) - f(x, y)| \leq T \\ 0 & \text{当 } |f(x_0, y_0) - f(x, y)| > T \end{cases}$$

5.2 SUSAN算子——原理

- 输出游程（run-length）和

$$S(x_0, y_0) = \sum_{(x, y) \in M(x, y)} C(x_0, y_0; x, y)$$

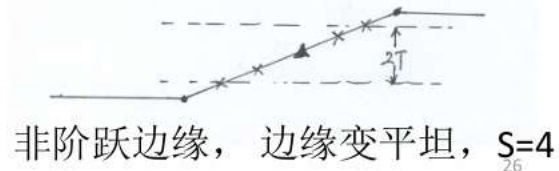
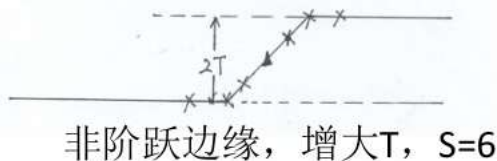
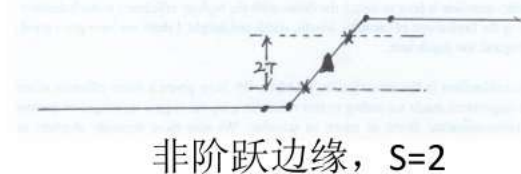
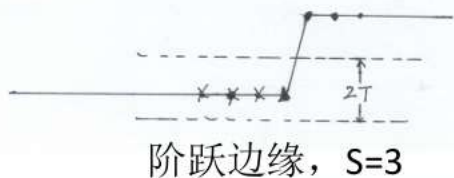
- 计算响应 $R(x_0, y_0)$

$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

几何阈值

5.2 SUSAN算子——原理

- 对于阶跃边缘， s 的值总会在某一边小于 $S_{\max}/2$? $\times$
- 对于非阶跃边缘， s 的值会更小，边缘检测不到的可能性更小? 取决于 T 和边缘的平滑度



5.2 SUSAN算子——原理

- 更稳定的计算 $C(.;.)$ 的公式

$$C(x_0, y_0; x, y) = \exp \left\{ - \left[\frac{f(x_0, y_0) - f(x, y)}{T} \right]^2 \right\}$$

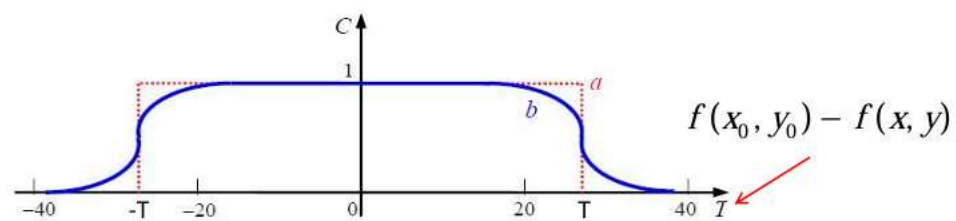
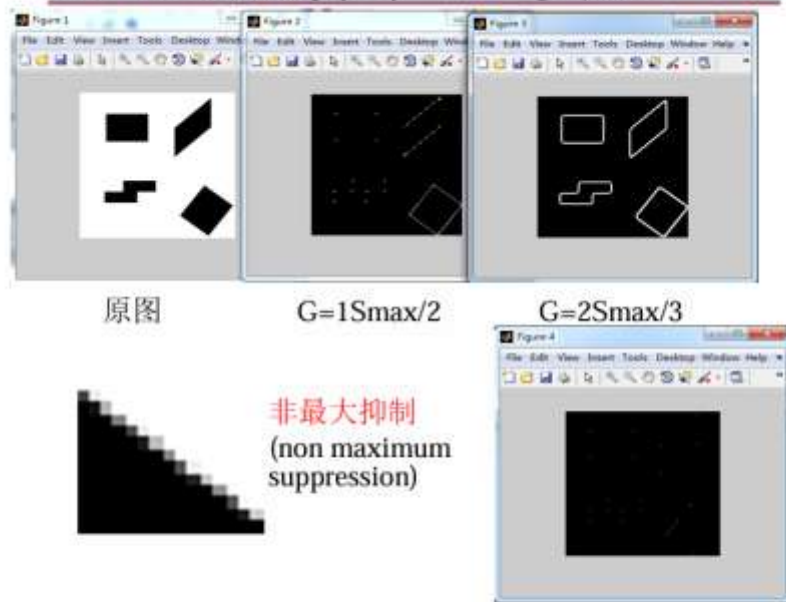


图 5.2.4 不同的函数 $C(.;.)$ 示例

5.2 SUSAN算子——实验



5.2 SUSAN算子——特点

- 无微分计算，抗噪性强
- 对边缘的响应随着边缘的平滑或模糊而增强
- 能提供不依赖于模板尺寸的边缘精度
- 控制参数（T和G）的选择很简单，且任意性较小

T 越小 $\rightarrow$ 核同值区越小 $\rightarrow$ 检测到的边缘点和角点越多

G越小 $\rightarrow$ 对检测得到的边缘点和角点要求越高 $\rightarrow$ 角点越尖

我们把 SUSAN 算子里做极大值抑制的具体动作拆解开，主要分为两种情况：

1. 针对角点：在 2D 九宫格里“卷”出最强者

当 SUSAN 算子在找角点（对应图中的 $G = 1S_{max}/2$ ）时，真实的物理角点附近的一小片像素，都会算出一个比较高的响应值（因为它们的 USAN 面积都很小）。如果不做抑制，图上就会出现“一坨”密集的点。

具体做法：

- 算法会以当前像素为中心，画一个 3×3 或 5×5 的局部小网格。
- 拿着当前像素的响应值，去和周围的这 8 个（或 24 个）邻居比大小。
- 只有当它的响应值是这片局部区域里的“绝对最大值”时，它才能活下来，被确认为最终的角点。
- 如果它比哪怕一个邻居小，直接被抹零（变黑）。这保证了最终每个角点都是孤立且精准的。

2. 针对边缘：沿着垂直方向寻找“山脊线”（图中楼梯效果的来源）

这正是你截图中左下角展示的场景。找边缘（对应图中的 $G = 2S_{max}/3$ ）比找角点要复杂一点。边缘是一条连续的线。如果你像找角点那样，直接在九宫格里找最大值，那边缘就会断裂成一个个孤立的点。真正的边缘非极大值抑制，必须带有方向性。

具体做法：

- 第一步：找到边缘的“法线方向”（垂直于边缘的方向）。
 - SIFT 或 Canny 算法通常用复杂的偏导数来算梯度方向。
 - **SUSAN 极其聪明：** 它利用已经算好的 USAN（核同值区）的**重心**。因为重心总是偏向远离边缘的一侧，所以**从中心像素指向重心的向量，恰好就垂直于边缘！**
- 第二步：沿法线方向比大小。
 - 计算机沿着这根刚刚找到的垂直法线，往前看一个像素，往后看一个像素。
 - 如果当前像素的响应值，大于它前面和后面的邻居，说明它站在边缘的“最高山脊”上，保留！
 - 如果它不如前后的邻居大（处于半山腰），直接将其响应值抹零（变黑）。

最终效果：

经过这种垂直方向的挤压，原本那条发虚、很宽的对角线边缘（图中左侧），就会被瞬间削薄，变成一条极其锐利、**只有一个像素宽**的精准边缘线（图中标注右侧的效果）。

为了让你直观看到这种“沿着垂直方向削薄边缘”的物理动作，我做了一个 **SUSAN 边缘非极大值抑制模拟器**。你可以点击运行，看着那条“发虚的粗边缘”是如何一步步被剥离成单像素直线的：

诶，那SUSAN检测角点和边缘的话就是设置不同的G吗？只要最好算出来的响应大于0就当角点了？



显示思路

你这两个问题问得非常刁钻！可以说是直接击穿了 SUSAN 算子的表象，直逼它的底层数学逻辑。

我分两个部分给你最直接的回答：

1. 测角点和测边缘，就只是设置不同的 G 吗？

绝大多数情况下，是的！ G 就是 SUSAN 算子区分角点和边缘的“物理开关”。

为什么这么神奇？我们回归到 SUSAN（最小核同值区）的物理定义上。假设我们有一个包含 37 个像素的圆形模板（ $S_{max} = 37$ ）。

- 在平坦区域：圆圈里几乎所有的像素都和中心像素一样，核同值区面积 $n \approx 37$ 。
- 在直线边缘上：边缘刚好把圆圈劈成两半，核同值区面积 $n \approx 18.5$ （也就是 $\approx S_{max}/2$ ）。
- 在直角角点上：角点只占了圆圈的四分之一，核同值区面积 $n \approx 9.25$ （也就是 $\approx S_{max}/4$ ）。

SUSAN 的响应公式是： $R = G - n$ （只有当 $n < G$ 时才算数，否则 $R = 0$ ）。

• 非极大值抑制（NMS）才是“总决赛”：

所有 $R > 0$ 的点，都只能叫“角点候选人”。

算法必须拿着这些候选人，在它们各自的 3×3 邻域里进行终极 PK（也就是上一页 PPT 里

做的动作）。只有那个站在真正的物理尖角上、 R 值绝对最大、傲视群雄的那个唯一像素，才

能被最终加冕为真正的角点。其他那些 $R > 0$ 但不是最大的跟班，全部被无情抹零。

• 如果你只想要角点（设置 $G = S_{max}/2$ ）：

- 遇到边缘时： $n \approx S_{max}/2$ ，响应值 $R \approx 0$ 。边缘被淘汰！
- 遇到角点时： $n \approx S_{max}/4$ ，响应值 $R > 0$ 。角点存活！

• 如果你既想要边缘又想要角点（设置 $G = 3S_{max}/4$ ）：

- 遇到边缘和角点，算出来的 R 都会大于 0，两者都能存活。

所以，通过无脑调小 G 的值，确实可以直接把边缘过滤掉，只留下极其尖锐的角点。

2. 只要算出来的响应 $R > 0$ ，就算角点了？

大错特错！这也是初学者最容易踩的坑！

如果只要 $R > 0$ 就算角点，那你又把我们上一回合讲的“非极大值抑制（NMS）”给忘了！

我们把这个过程比作一场选秀：

• 计算 $R > 0$ 只是“海选”：

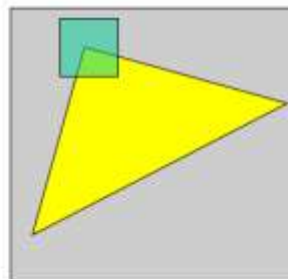
当你把圆形模板放在物理角点的正中心时， n 最小， R 最大。

但是，当你把模板稍微偏离角点 1 个像素时，虽然 n 变大了一点点，但它依然小于 G ，算出来的 R 依然大于 0！

这就是为什么如果你只看 $R > 0$ ，你的画面上会出现一大坨密密麻麻的白点（角点扎堆），或者一条很粗的马路（边缘发虚）。

5.3 Harris算子——原理

- 基本思想
 - 通过一个小的滑动窗口检测角点
 - 若窗口以角点为中心，那么在任意方向上平移该窗口，窗口内的灰度值都将会有剧烈的变化

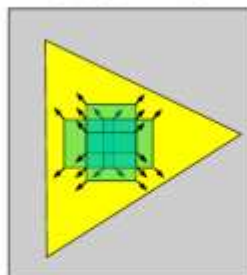


59

5.3 Harris算子——原理

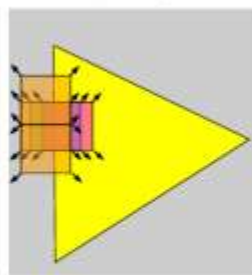
- 基本思想

平滑区域



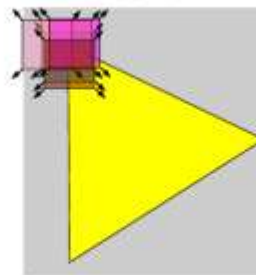
各个方向
基本不变

边缘



边缘方向
基本不变

角点



各个方向均
有剧烈变化

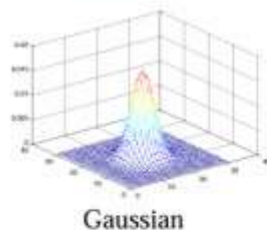
5.3 Harris算子——原理

- 设平移量为 (u, v) ，用图像的自相关函数表示窗口内灰度值的变化

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

↑↑↑
窗口函数平移后的灰度值灰度值

窗口函数 $w(x, y) =$



61

5.3 Harris算子——原理

- 用Taylor展开近似任意方向

$$\begin{aligned} E(u, v) &= \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2 \\ &= \sum_{x, y} w(x, y) [I_x u + I_y v + o(u^2, v^2)]^2 \end{aligned}$$

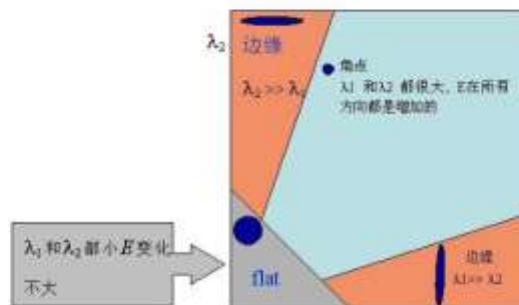
- 表达为矩阵形式

$$E(u, v) \cong [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$
$$M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

62

5.3 Harris算子——原理

- 计算矩阵 M 的两个特征值
- 若两个都很大就是角点，一大一小就是边缘，两个都小就是在变化缓慢的图像区域



63

5.3 Harris算子——原理

- 利用矩阵 M 两个特征值的和等于 M 的迹，两个特征值的积等于 M 的行列式，计算Harris角点响应值

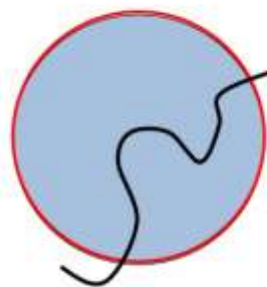
$$R = \det M - k(\text{Trace}M)^2 \quad k = 0.04 \sim 0.06$$

- 响应值大于阈值的点为候选点
- 非最大抑制去除干扰

64

角点自动尺度选择

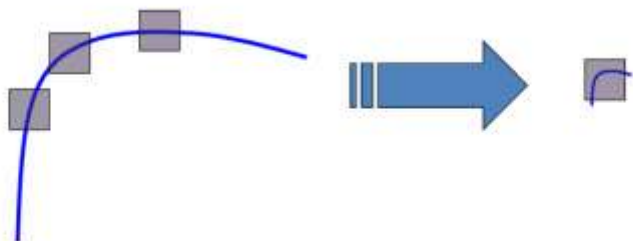
- 尺度变化检测
 - 角点应该在尺度和位置上都是变化最大



73

角点自动尺度选择

- 基于固定窗口的角点检测方法对尺度敏感



大尺度上可能检测为边缘

小尺度上为角点!

74

角点自动尺度选择

Lindeberg et al., 1996



$f(I_{x,y})(x, \sigma)$

from Tinne Tuytelaars

角点自动尺度选择



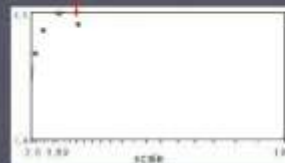
$f(I_{x,y})(x, \sigma)$

角点自动尺度选择



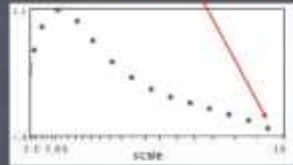
$f(I_{x,y})(x, \sigma)$

角点自动尺度选择



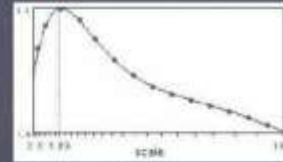
$f(I_{x,y})(x, \sigma)$

角点自动尺度选择



$$f(I_{k-\omega}(x, \sigma))$$

角点自动尺度选择

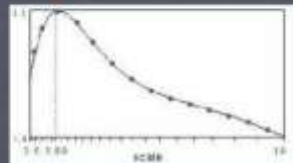


$$f(I_{k-\omega}(x, \sigma))$$



$$f(I_{k-\omega}(x', \sigma'))$$

角点自动尺度选择



$$f(I_{k-\omega}(x, \sigma))$$

Automatic scale selection

Normalize: rescale to fixed size



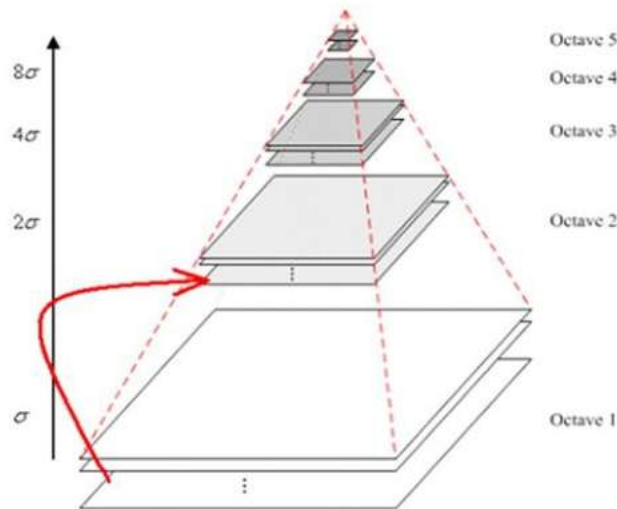
5.4 SIFT算子——原理

- 融特征点检测与描述于一体
- 基本步骤：
 - 1、尺度空间极值检测
 - 2、关键点定位
 - 3、获取关键点主方向
 - 4、关键点描述

85

5.4 SIFT算子——原理

- 尺度空间极值检测
 - 高斯金字塔构建



每一级octave分为 S 个尺度间隔，相邻两个尺度的 σ 之比为：

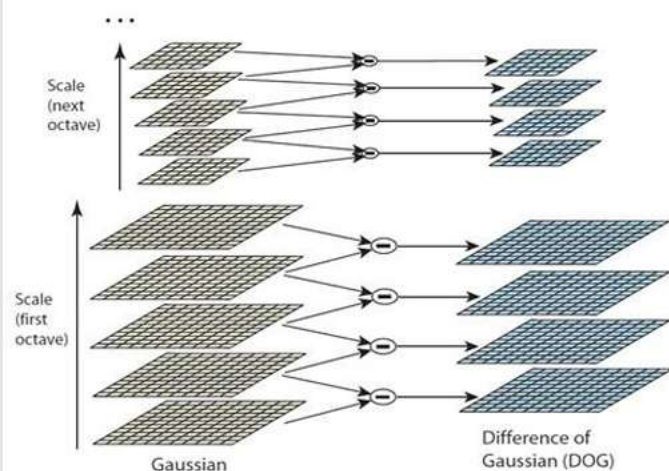
$$k = 2^{1/S}$$

则每一级包含 $1, 2^{1/S}, 2^{2/S}, \dots, 2$ ，共 $(S+1)$ 幅图像

88

5.4 SIFT算子——原理

- 尺度空间极值检测
 - DoG金字塔计算



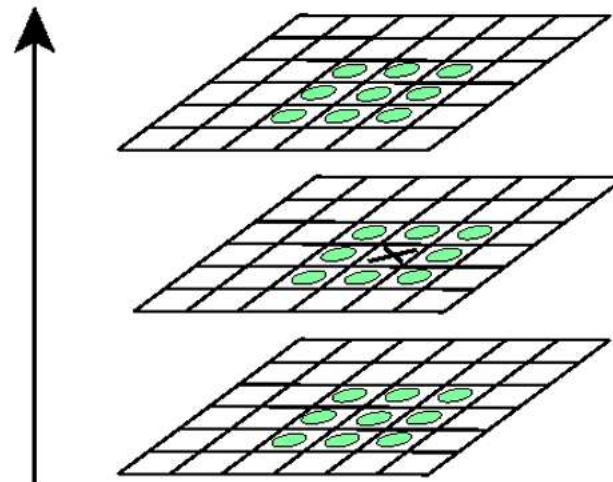
$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

选择DoG金字塔的原因:

- 1、计算简单
- 2、提供了尺度归一化Laplacian of Gaussian (LoG) 算子的良好近似 (参考Lindeberg (1994))。而LoG算子的极值, 相对于梯度、Hessian、Harris等算子, 能提供更稳定的图像特征 (参考Mikolajczyk (2002))

5.4 SIFT算子——原理

- 尺度空间极值检测
 - 空间极值点检测



对DoG金字塔中的每一像素, 在三维邻域(二维图像、一维尺度)内判断是否极值点

可以确定极值点的位置及尺度

极值的大小反映了极值处的对比度(反差)

为得到每一级的 s 个极值结果, 高斯金字塔的每一级需有 $(s+3)$ 幅图像

5.4 SIFT算子——原理

- 关键点定位

- 亚像素（子像素）精确定位

将原点移到极值点处，并将高斯差分在该极值点处展开，保留一阶项及二阶项，得到：

$$D(\mathbf{x}) = D + \frac{\partial D^T}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

其中 $\mathbf{x} = (x, y, \sigma)^T$

令 $D'(\mathbf{x}) = 0$ ，得：

$$\hat{\mathbf{x}} = -\frac{\partial^2 D^{-1}}{\partial \mathbf{x}^2} \frac{\partial D}{\partial \mathbf{x}}$$

尺度

94

5.4 SIFT算子——原理

- 关键点定位

- 去除边缘点

由于DoG算子对边缘点也很敏感，在边缘处也有很大的响应值。利用Hessian矩阵来处理边缘点。

在极值点处，利用 $D(\mathbf{x})$ 计算Hessian矩阵

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{yx} & D_{yy} \end{bmatrix}$$

$D(\mathbf{x})$ 的主曲率，在边缘梯度的方向上比较大，而在沿着边缘的方向上则较小，且与 H 的特征值成正比。

96

5.4 SIFT算子——原理

- 关键点定位

- 去除低对比度极值点

利用 $\hat{\mathbf{x}}$ 处的高斯差分 $D(\hat{\mathbf{x}})$ ，去除对比度较低的不稳定极值点

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{x}} \hat{\mathbf{x}}$$

设对比度阈值为 T ，则当 $|D(\hat{\mathbf{x}})| < T$ 时，去除极值点 $\hat{\mathbf{x}}$

95

5.4 SIFT算子——原理

- 关键点定位

- 去除边缘点

利用矩阵 H 两个特征值的和等于 H 的迹，两个特征值的积等于 H 的行列式，有

$$\frac{\text{Tr}^2(H)}{\det(H)} = \frac{(r+1)^2}{r}$$

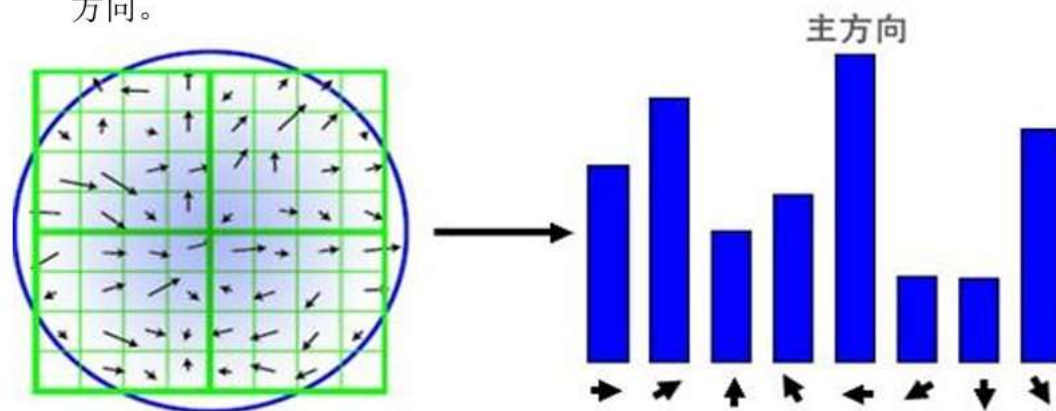
其中 r 为 H 的大特征值与小特征值之比

设关于 r 的阈值为 T_r ，则当 $\frac{\text{Tr}^2(H)}{\det(H)} > \frac{(T_r+1)^2}{T_r}$ 时，判断该极值点为边缘点，并去除

5.4 SIFT算子——原理

- 获取关键点主方向

根据关键点的尺度，在高斯金字塔对应图像的关键点位置处的局部区域计算梯度方向直方图。直方图峰值对应得方向即为关键点的主方向。



基于梯度方向的线性插值：避免梯度方向直方图在梯度方向量化的边界处的突然变化

5.4 SIFT算子——原理

- 关键点描述

- 确定用于计算关键点描述的局部区域

描述子梯度方向直方图由关键点所在尺度的模糊图像计算产生，局部区域的半径通过下式计算：

$$\text{radius} = \frac{3\sigma_{\text{ocr}} \times \sqrt{2} \times (d+1) + 1}{2} \quad ?$$

其中 σ_{ocr} 为关键点所在组（octave）的组内尺度, $d=4$

$$\text{diameter} = 6\sigma + 1 \quad \checkmark$$

其中 σ 为关键点所在尺度

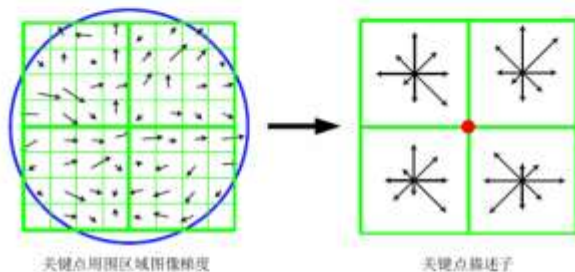
102

5.4 SIFT算子——原理

- 关键点描述

- 计算关键点描述

在局部区域内对每个像素点求其梯度幅值和方向，生成方向直方图



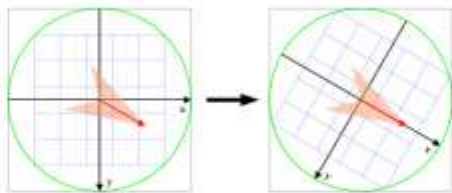
梯度幅值高斯（ σ 为局部区域大小的一半）加权：离关键点越近/越远，其对关键点描述的作用越大/越小
基于像素位置和梯度方向的三线性插值：避免梯度方向直方图在分块边界和梯度方向量化的边界处的突然变化

104

5.4 SIFT算子——原理

- 关键点描述

- 根据关键点主方向变换坐标系



5.4 SIFT算子——原理

- 关键点描述

- 归一化关键点描述

描述子向量归一化：克服光照影响，主要是增益的影响

方向直方图每个方向上梯度幅值限制在一定门限值以下（门限一般取0.2，像素灰度值在0~1范围内）：克服相机饱和、光照对3D表面不同方位影响不同等非线性光照情形

5.5 SURF算子——原理

- 特征点检测

- 基于Hessian矩阵的特征点检测
- 基于盒子（Box）滤波器的Hessian矩阵近似计算
- 基于积分图的盒子滤波器快速计算
- 尺度金字塔构建
- 特征点定位

- 特征点描述

- 快速索引匹配

5.5 SURF算子——原理

- 特点:

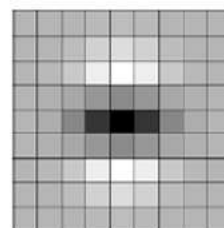
- 具有尺度和旋转不变性特点的特征点提取和描述方法
- 可重复性更好
- 鲁棒性更强
- 计算更快

5.5 SURF算子——原理

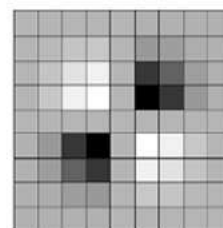
- 基于Hessian矩阵的特征点检测
 - 给定输入图像 I 中一点 $\mathbf{x}=(x, y)$ ，尺度为 σ 的Hessian矩阵在该点的值 $H(\mathbf{x}, \sigma)$ 表示如下：
$$H(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$
其中， $L_{xx}(\mathbf{x}, \sigma)$ 为高斯二阶偏导数 $\frac{\partial^2}{\partial x^2} g(\sigma)$ 与输入图像 I 在点 $\mathbf{x}$ 处的卷积，其余类似
 - 行列式的值即为点 $\mathbf{x}$ 处的响应

5.5 SURF算子——原理

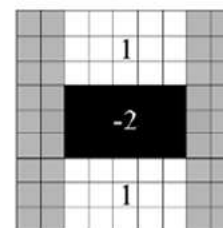
- 基于盒子滤波器的Hessian矩阵近似计算
 - 由于实际中高斯二阶导函数的离散化和截断造成的误差是不可避免的，近似为盒子（box）滤波以降低计算量



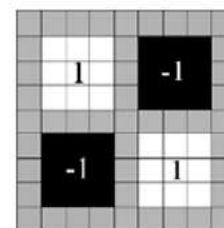
高斯二阶偏导数
 L_{yy}



高斯二阶偏导数
 L_{xy}



盒子滤波近似函数
 D_{yy}



盒子滤波近似函数
 D_{xy}

5.5 SURF算子——原理

- 基于盒子滤波器的Hessian矩阵近似计算

– 近似Hessian矩阵行列式可表示为:

$$\det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

– 其中权重w为平衡因子

$$w = \frac{|L_{xy}(1.2)|_F |D_{yy}(9)|_F}{|L_{yy}(1.2)|_F |D_{xy}(9)|_F} = 0.912... \approx 0.9$$

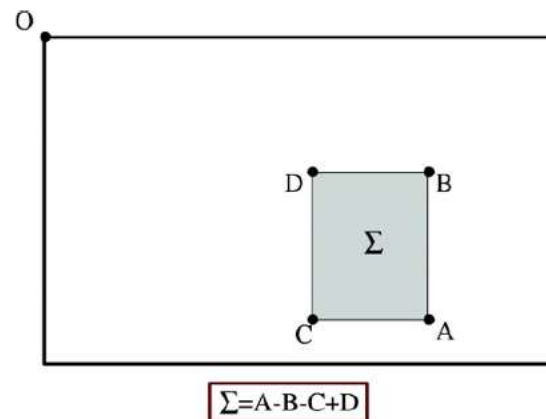
Frobenius
范数

– 行列式的值即为点x处的响应

5.5 SURF算子——原理

- 基于积分图的盒子滤波器快速计算

– 计算矩阵ABCD中像素和只需要积分图中A、B、C、D四点的值和3次加法运算



5.5 SURF算子——原理

- 尺度金字塔构建

- 根据不同尺度，构建多组尺度金字塔
- 每组尺度金字塔的各层尺度成等差数列
- 不同组的尺度金字塔的公差等比增长

$$\text{Filter Size} = 3(2^{\text{Octave}} \times \text{Scale} + 1)$$

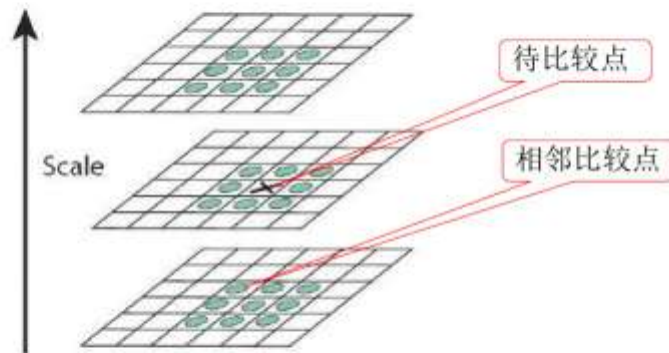
- 为保持尺度空间的连续性，相邻组尺度金字塔有部分层重叠

124

5.5 SURF算子——原理

- 特征点定位

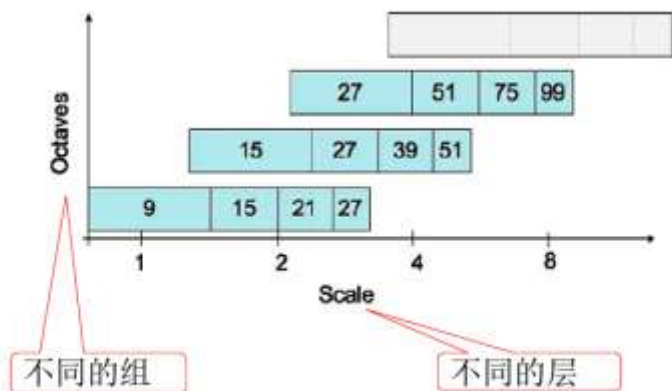
- 非极大抑制



127

5.5 SURF算子——原理

- 尺度金字塔构建



125

5.5 SURF算子——原理

- 特征点定位

- 尺度空间插值
- 得到亚像素级的特征点
- 减小不同组第一层之间的差异

128

5.5 SURF算子——原理

- 特征点主方向
 - 选择以特征点为中心、半径为 $6s$ （ s 为特征点所在的尺度值）的圆形邻域
 - 计算该圆形邻域内各点的 x 、 y 方向Haar小波响应（Haar小波边长取 $4s$ ）

131

5.5 SURF算子——原理

- 特征点主方向
 - 用 $\sigma=2.5s$ 的高斯函数对响应加权
 - 以特征点为中心，张角为 $\pi/3$ 的扇形滑动窗口，计算窗口内的Harr小波响应值 dx 、 dy 的累加

$$m_w = \sum_w dx + \sum_w dy$$
$$\theta_w = \arctan\left(\sum_w dx / \sum_w dy\right)$$

$$s = 1.2 \times \frac{\text{Filter Size}}{9}$$

133

5.5 SURF算子——原理

- 特征点主方向
 - Haar小波模板，白色权重为1，黑色为-1



X方向Haar小波滤波器



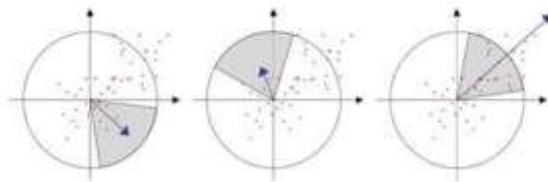
Y方向Haar小波滤波器

132

5.5 SURF算子——原理

- 特征点主方向
 - 特征点主方向为最大的Harr响应累加值所对应的方向

$$\theta = \theta_w | \max\{m_w\}$$



134

5.5 SURF算子——原理

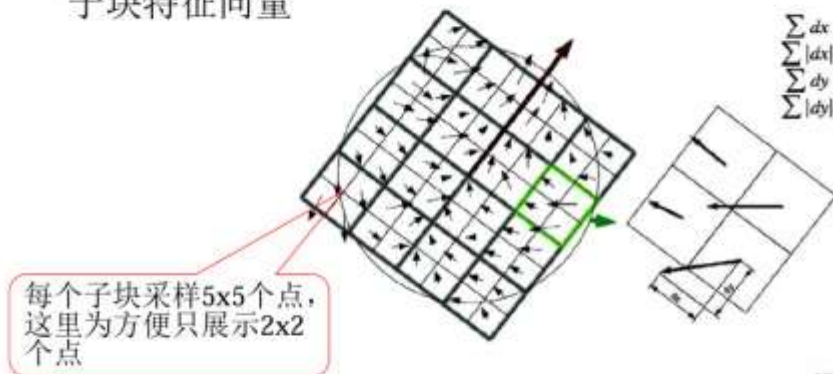
- Haar小波响应和描述符
 - 构建以特征点为中心、方向为特征点主方向、边长为 $20s$ 的正方形区域



1.35

5.5 SURF算子——原理

- Haar小波响应和描述符
 - 在各子块内统计 $\sum dx$ 、 $\sum |dx|$ 、 $\sum dy$ 、 $\sum |dy|$ 形成子块特征向量



1.37

5.5 SURF算子——原理

- Haar小波响应和描述符
 - 将区域划分为 4×4 个子块
 - 每个子块内均匀采样 5×5 个样本点
 - 计算各采样点的Haar小波响应（Haar小波边长为 $2s$ ）
 - 用以特征点为中心点， $\sigma=3.3s$ 的高斯函数对响应加权

1.36

5.5 SURF算子——原理

- Haar小波响应和描述符
 - 每个子块统计一个4维特征向量 v
$$v = (\sum dx, \sum dy, \sum |dx|, \sum |dy|)$$
 - 将 4×4 个子块的特征向量 v 相连可得长度为64的特征向量，具有亮度不变性
 - 将特征向量归一化，具有对比度不变性

1.38

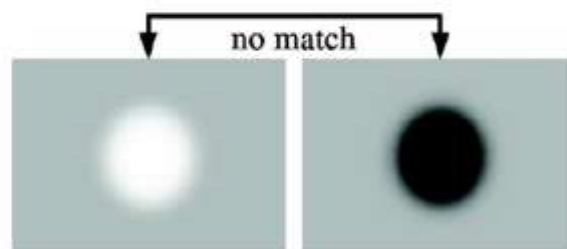
5.5 SURF算子——原理

- 快速索引匹配
 - 特征点一般表现为斑点（blob）结构
 - 根据拉普拉斯算子的符号（即Hessian矩阵的迹的符号）可区分亮斑点和暗斑点

141

5.5 SURF算子——原理

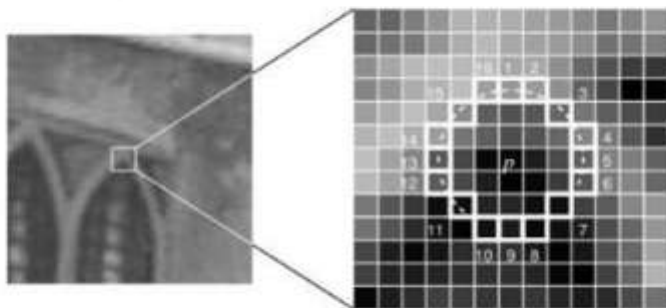
- 快速索引匹配
 - 在检测阶段，记录拉普拉斯算子的符号
 - 在匹配阶段中，匹配拉普拉斯算子符号相同的特征，从而降低运算量



142

5.6 FAST算子——原始算法

- 在像素 p 的Bresenham圆上选择 N 个像素。如果这 N 个像素中存在 n 个连续像素的灰度值都高于 I_p+t ，或低于 I_p-t ，则 p 是角点，其中 I_p 为 p 的灰度值， t 为阈值。

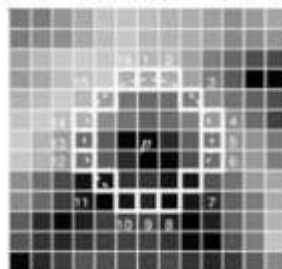


$N=16$ 示意图（以后均以此为例）

156

5.6 FAST算子——原始算法

- 加速（以 $n=12$ 为例）：
 - 检测像素点周围的四个点（1, 5, 9, 13）中是否有三个点满足阈值要求
 - 如果不满足，则为非角点
 - 如果满足，则判断16个点中是否至少有12个连续点满足条件。若满足，则为角点，否则为非角点



160

5.6 FAST算子——原始算法

- 与SUSAN算子对比

SUSAN算子

FAST算子

T : 灰度差阈值
(不区分亮暗)

t : 灰度差阈值
(区分亮暗)

G : 几何阈值
(控制同值像素数)

n : 几何阈值
(控制非同值像素数)

考虑模板区域

考虑模板边界

无同值区域连通要求

要求非同值像素连续

5.6 FAST算子——原始算法

- 不足：
 - 1、 $n < 12$ 时无法使用加速算法
 - 2、计算效率依赖于角点表现的分布和查询Bresenham圆上像素灰度值的查询顺序
 - 3、检测到的很多角点都是连在一起的
- 解决途径：
 - 1、机器学习
 - 2、非最大抑制（Non Maximum Suppression, NMS）

E. Rosten, T. Drummond, Machine learning for high-speed corner detection, ECCV 2006.

161

5.6 FAST算子——改进算法

• 机器学习:

- 1、给定训练图像集合, 设定 n 和阈值 t
- 2、用前述角点检测算法检测出训练图像集中所有的角点
- 3、对于像素 p , 其Bresenham圆上的像素 $p \rightarrow x$, $x \in \{1, 2, \dots, 16\}$, 有如下三种状态:

$$S_{p \rightarrow x} = \begin{cases} d & I_{p \rightarrow x} \leq I_p - t & \text{暗} \\ s & I_p - t < I_{p \rightarrow x} < I_p + t & \text{相似} \\ b & I_{p \rightarrow x} \geq I_p + t & \text{亮} \end{cases}$$

167

5.6 FAST算子——改进算法

• 机器学习:

- 6.1、计算集合 P 的 K_p 熵

$$H(P) = (c + \bar{c}) \log_2 (c + \bar{c}) - c \log_2 c - \bar{c} \log_2 \bar{c}$$

其中

$$c = |\{p | K_p \text{ 为真} \}|$$

$$\bar{c} = |\{p | K_p \text{ 为假} \}|$$

即 c 和 $\bar{c}$ 分别表示训练图像集中角点和非角点的像素数

角点和非角点各占一半时, 熵最大:

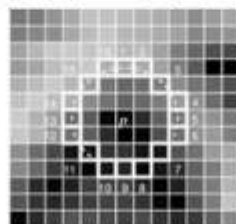
全为角点或非角点时, 熵最小, 为0

169

5.6 FAST算子——改进算法

• 机器学习:

- 4、给定一个 x , 对所有的像素 $p \in P$ (训练图像集中的所有像素), 计算 $S_{p \rightarrow x}$, 从而将 P 划分为 P_d 、 P_s 、 P_b 三个集合
- 5、定义一个布尔变量 K_p , 当像素 p 为角点时, K_p 为真, 否则为假
- 6、若将 x 视为特征, 则可采用ID3算法构建决策树



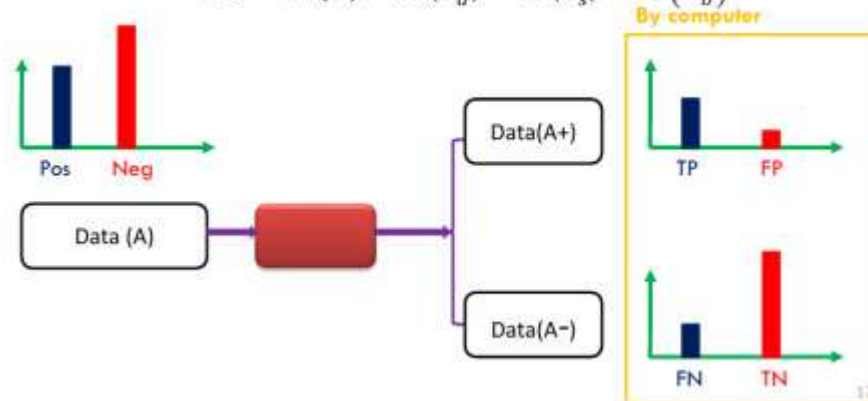
168

5.6 FAST算子——改进算法

• 机器学习:

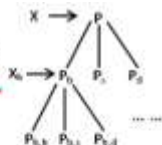
- 6.2、选择使得信息增益最大的 x

$$IG = H(P) - H(P_d) - H(P_s) - H(P_b)$$



170

5.6 FAST算子——改进算法



• 机器学习:

- 6.3、对 P_d 、 P_s 、 P_b 迭代重复上述过程，例如选择 x_b 将 P_b 划分为 $P_{b,d}$ 、 $P_{b,s}$ 、 $P_{b,b}$ ，选择 x_s 将 P_s 划分为 $P_{s,d}$ 、 $P_{s,s}$ 、 $P_{s,b}$ ，直至某一子集的熵为0（子集中的像素全为角点或全不为角点）
- 6.4、为了进一步加速，限制 x_b 、 x_s 、 x_d 必须是相同的，因此第二次判断时的像素是相同的，于是第一次判断和第二次判断可以使用许多高性能微处理器都拥有的向量指令并行处理。由于大多数像素经过两次判断后就被排除，因此极大地提高了速度。

172

5.6 FAST算子——改进算法

• 非最大抑制:

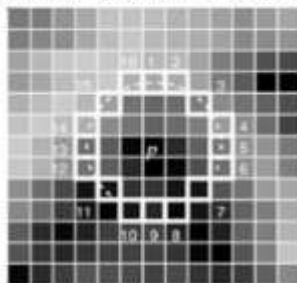
- 1、构建响应（得分）函数V
- 2、用V对每个检测到的角点打分
- 3、若角点a有一个得分更高的邻居b，则将其删除

174

5.6 FAST算子——改进算法

• 机器学习:

- 机器学习得到的角点检测器取决于训练样本。若训练样本未包含所有可能情况，该角点检测器与原始的采用分段测试（segment test）的检测器不会完全相同。这可以通过生成满足分段测试情况的所有可能样本来得到。



173

5.6 FAST算子——改进算法

• 非最大抑制:

- 关键是构建响应（得分）函数V
 - 三种定义方式（1）使得p仍为角点的最大n值；（2）使得p仍为角点的最大t值；（3）p与其Bresenham圆上连续亮或暗像素灰度值绝对差之和
 - 定义（1）和（2）容易导致很多像素拥有相同的得分值
 - 使用定义（3）的简化版（不要求连续）：

$$V = \max \left(\sum_{x \in S_{\text{light}}} |I_{p \rightarrow x} - I_p| - t, \sum_{x \in S_{\text{dark}}} |I_p - I_{p \rightarrow x}| - t \right)$$

$$S_{\text{bright}} = \{x | I_{p \rightarrow x} \geq I_p + t\} \quad S_{\text{dark}} = \{x | I_{p \rightarrow x} \leq I_p - t\}$$

175

作业

- 5.6

5.6 如果用排成 5 行的 13 个像素来近似表示圆形模板 (各行分别有 1, 3, 5, 3, 1 个像素), 这相当于一个半径约为多少个像素的圆? 此时灰度差的阈值应选多少?

几何

作业

- 请针对如下的 6 个分属于两类的二维特征样本, 分析采用哪一维特征进行分类, 可以获得最大的信息增益。

类别1: (2, 1), (2, 2), (4, 3)

类别2: (2, 3), (2, 4), (4, 2)

作业

- SIFT算子在计算关键点描述时, 使用了基于像素位置和梯度方向的三线性插值。请针对其中的梯度方向插值, 解释其作用, 并举例说明。假设梯度方向划分为0度、45度、90度、135度、180度、225度、270度、315度等8个方向。
- 请针对下图, 给出积分图的每一步计算式及计算结果, 并给出由积分图计算中间黑色阴影区域像素之和的计算式及结果。

3	2	4	1	7
2	8	7	0	1
4	2	8	4	2
5	6	5	5	1
2	3	7	6	9